

Experiment - 7

Verifying the Performance of Image Pre-processing with Otsu's Function using Python

✓ AIM

To verify the performance of image preprocessing using **Otsu's thresholding**, which automatically finds the optimal threshold for separating foreground and background in a grayscale image.

✓ ALGORITHM

1. Create or load a grayscale image.
2. Flatten image pixels into a 1-D array.
3. Compute histogram for pixel intensities (0–255).
4. For each threshold t from 0 to 255:
 - Compute class probabilities w_0, w_1 .
 - Compute class means m_0, m_1 .
 - Compute **between-class variance**

$$\sigma_b^2 = w_0 w_1 (m_0 - m_1)^2$$

5. Choose the threshold with the **maximum between-class variance**.
6. Apply this threshold to convert image $\rightarrow$ **binary image**.
7. Print threshold and binary image (performance verification).

✓ PYTHON CODE

```
main.py  Run  Clear
1 import numpy as np
2
3 # -----
4 # 1. Create a small synthetic grayscale image (8x8)
5 # -----
6 image = np.array([
7     [10, 12, 15, 14, 200, 205, 198, 210],
8     [11, 13, 17, 16, 202, 207, 199, 215],
9     [9, 10, 14, 13, 195, 200, 190, 205],
10    [8, 12, 15, 11, 198, 203, 195, 210],
11    [50, 55, 60, 58, 180, 185, 175, 190],
12    [52, 57, 62, 59, 182, 187, 179, 192],
13    [48, 53, 58, 55, 170, 175, 168, 180],
14    [49, 54, 59, 56, 172, 178, 170, 185],
15 ])
16
17 # -----
18 # 2. Otsu Threshold Implementation
19 # -----
20 def otsu_threshold(img):
21     pixels = img.flatten()
22     hist = np.bincount(pixels, minlength=256)
23     total = len(pixels)
24
25     sum_total = np.dot(np.arange(256), hist)
26     sum_background = 0
27     weight_background = 0
28
29     max_variance = 0
30     threshold = 0
31
32     for t in range(256):
```

```
Original Image:
[[ 10  12  15  14 200 205 198 210]
 [ 11  13  17  16 202 207 199 215]
 [  9  10  14  13 195 200 190 205]
 [  8  12  15  11 198 203 195 210]
 [ 50  55  60  58 180 185 175 190]
 [ 52  57  62  59 182 187 179 192]
 [ 48  53  58  55 170 175 168 180]
 [ 49  54  59  56 172 178 170 185]]

Computed Otsu Threshold: 62

Binary Image After Otsu Thresholding (0 = background, 1 = object):
[[0 0 0 0 1 1 1 1]
 [0 0 0 0 1 1 1 1]
 [0 0 0 0 1 1 1 1]
 [0 0 0 0 1 1 1 1]
 [0 0 0 0 1 1 1 1]
 [0 0 1 0 1 1 1 1]
 [0 0 0 0 1 1 1 1]
 [0 0 0 0 1 1 1 1]]

=== Code Execution Successful ===
```

✓ RESULT

The Otsu's thresholding algorithm successfully computed an optimal threshold (around **95** for the synthetic image).

Pixels below the threshold were classified as **background (0)** and pixels above as **foreground (1)**.

This verifies the performance and correctness of Otsu's image preprocessing technique.